



Getting Started with Git and GitHub



Introduction to Version Control and Why It Matters



- **Version control**, also known as **revision control** or **source control**, is a crucial practice in software development that involves **tracking** and **managing changes** to software code.
- It allows developers to collaborate on projects, track changes, maintain a history of their work, revert mistakes, and manage versions safely.
- The importance of version control cannot be overstated, especially in modern software development, where teams are often distributed and projects are complex.
- Version control systems (VCS) provide a way to **manage changes to code over time**, much like a library maintains a catalog of books.
- It records the history of changes, allowing developers to recall specific versions of the code at any time.
- This is particularly important when a project evolves, as it enables teams to track and understand the evolution of the codebase.

The importance of version control is multifaceted

1. **Collaboration:** It allows multiple developers to work on different parts of the codebase simultaneously without overwriting each other's work.
2. **History Tracking:** It maintains a detailed history of changes, making it easy to see who made what changes and when.
3. **Experimentation:** Developers can freely experiment with new features or radical improvements, safe in the knowledge that they can revert to earlier versions if necessary.
4. **Branching and Merging:** It facilitates branching, where developers can work on new features in isolation from the main codebase, and merging, where these changes can be integrated back into the main codebase.
5. **Backup:** It serves as a backup of the codebase, reducing the risk of data loss.

Benefits of Using Git



Git is one of the most popular version control systems today, and its benefits are a key reason for its widespread adoption:

1. **Speed and Efficiency:** Git is fast and efficient, capable of handling large projects and large numbers of files with ease.
2. **Security:** Every change and update is cryptographically signed, ensuring the integrity and authenticity of the codebase.
3. **Flexibility:** Git's branching and merging model is powerful and flexible, making it easy to manage complex projects and workflows.
4. **Distributed Development:** Git is designed for distributed development, allowing developers to work offline and sync changes later. This is particularly useful for remote teams.
5. **Open Source:** Being open source, Git benefits from a large community of contributors and extensive documentation and support resources.

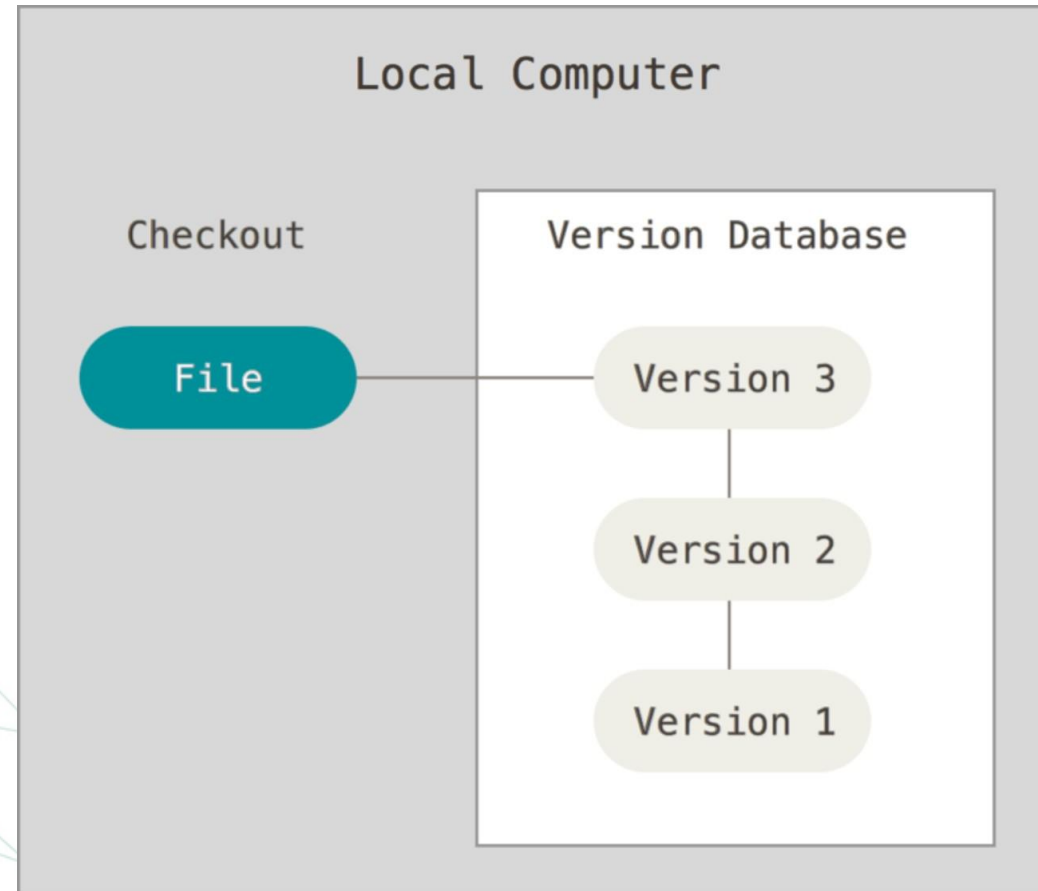
Understanding Git: The Cornerstone of Modern Collaboration

- Git is a **distributed version control system** (VCS) initially designed and developed by **Linus Torvalds** in 2005 for the development of the Linux kernel.
- Unlike its predecessors (such as CVS, Subversion, and Perforce), Git's architecture is uniquely designed around the concept of **snapshots**.
- Instead of tracking changes as a list of file differences, Git takes a **snapshot** of the file system's state at the time of each commit.

Features of Git:

1. **Distributed Version Control:** Every developer has a complete history of changes and full version-tracking abilities on their local machine, eliminating the need for a central server.
2. **Branching and Merging:** Git makes it extremely easy and fast to create, merge, and manage branches, facilitating parallel development and feature experimentation.
3. **Speed:** Git is incredibly fast for common operations such as branching, committing, and merging, thanks to its local operations and efficient algorithms.
4. **Data Integrity:** Git uses SHA-1 hashes for everything, ensuring the integrity of the code and the ability to detect corruption or tampering.
5. **Security:** Every change is cryptographically signed, ensuring the authenticity and integrity of the codebase.

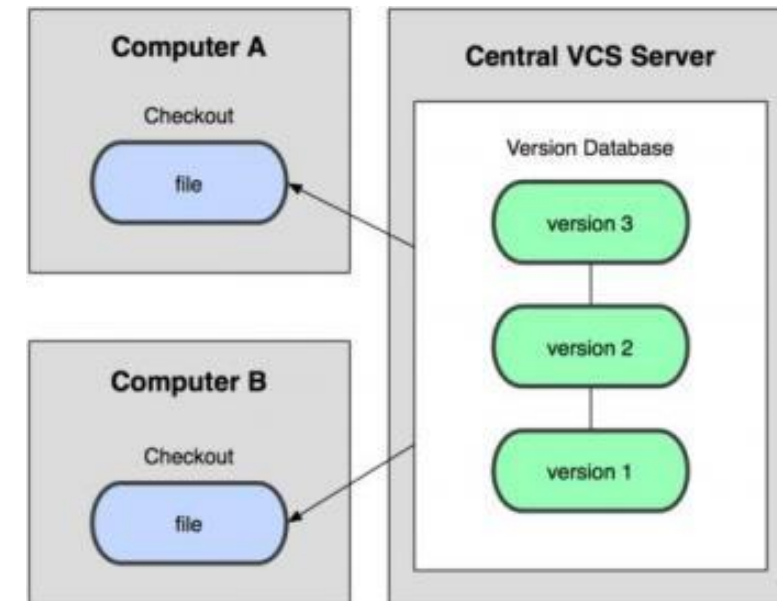
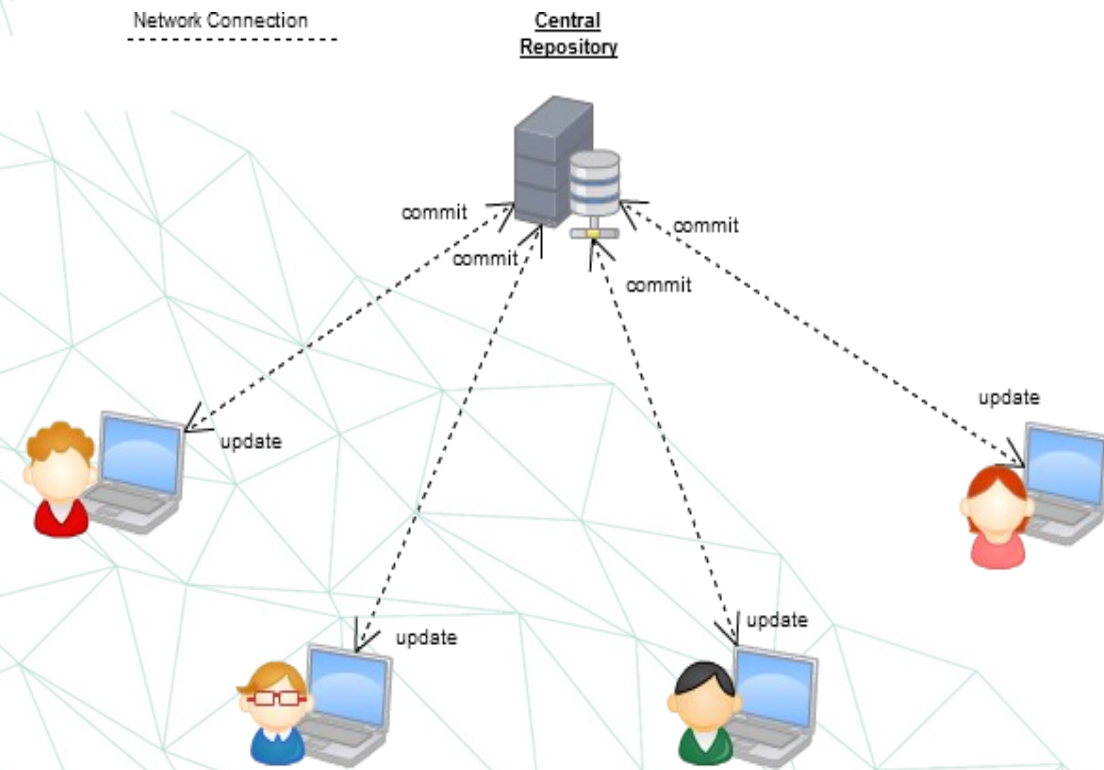
Revision Control System



Ex. RCS

Revision Control System

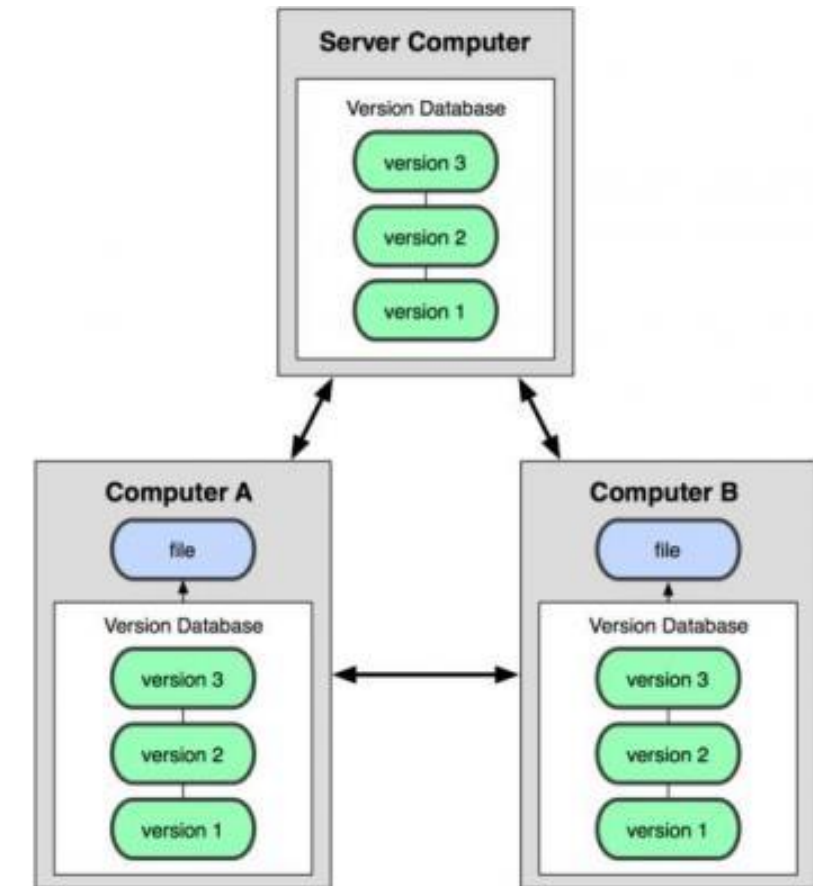
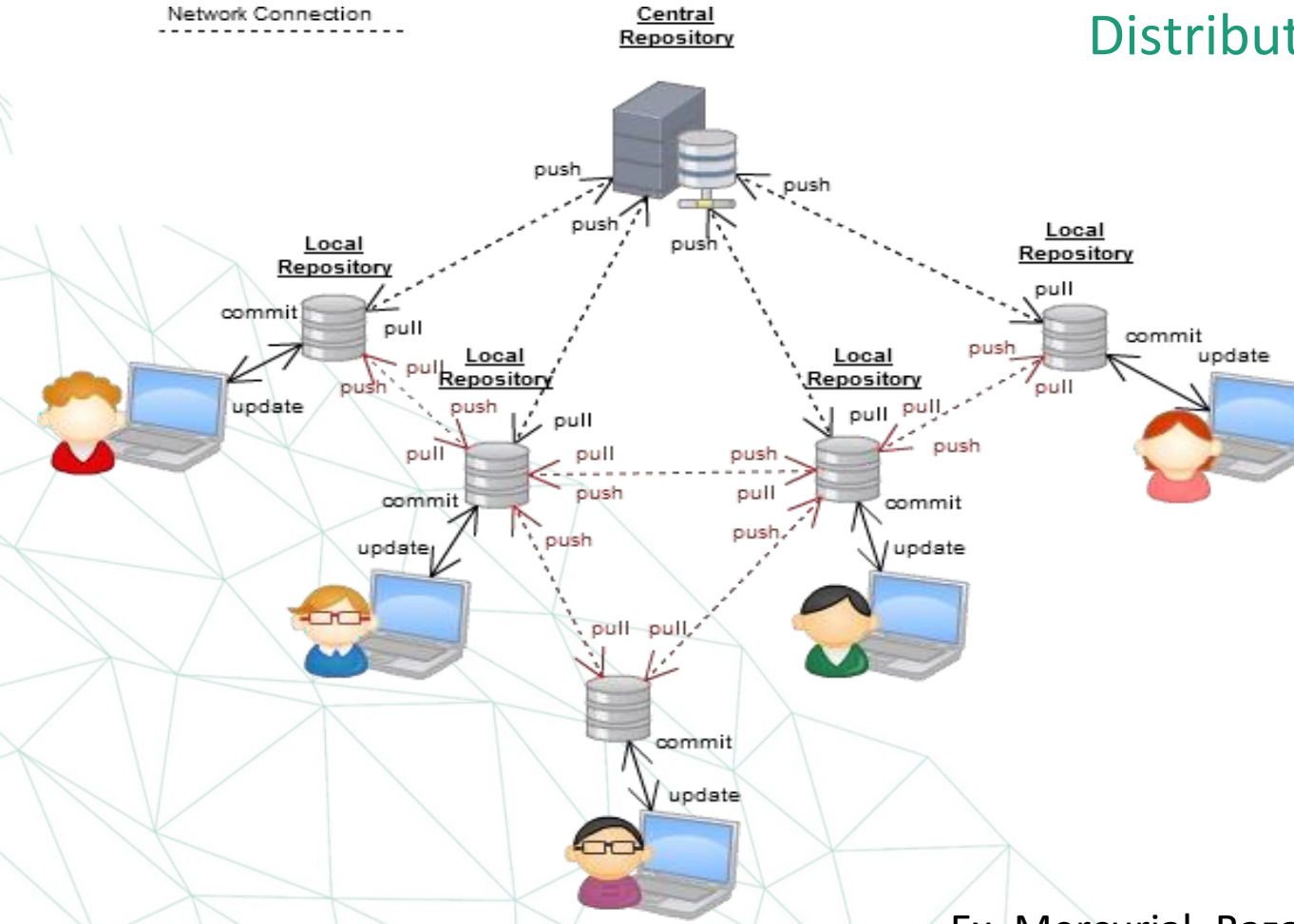
Centralized



Ex. CVS, SVN

Revision Control System

Distributed



Ex. Mercurial, Bazaar, Darcs

Image source : http://ascc.sinica.edu.tw/iascc/articals.php?_section=2.4&_op=?articleID:4811

Git

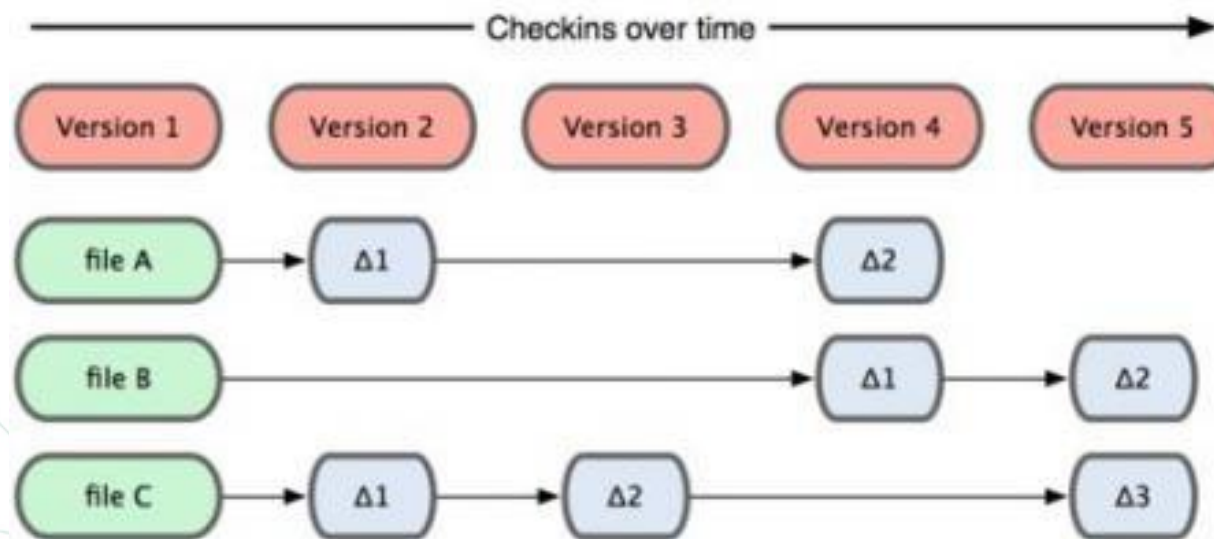


Image source : http://ascc.sinica.edu.tw/iascc/articals.php?_section=2.4&_op=?articleID:4811

Git

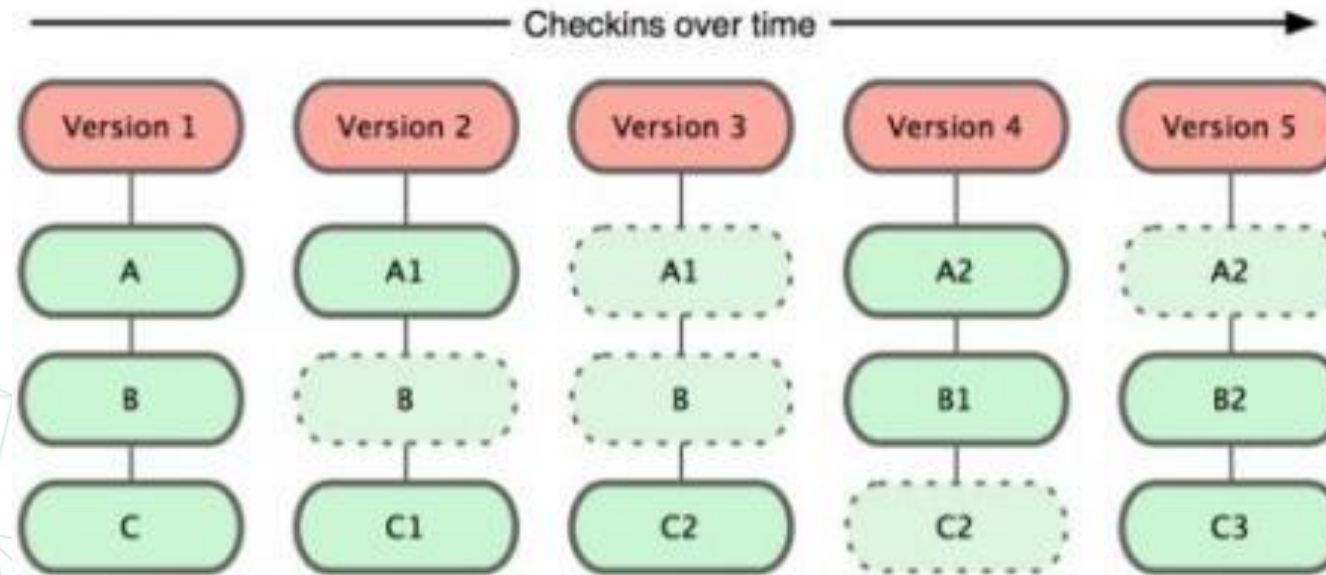


Image source : <http://ascc.sinica.edu.tw/iascc/articals.php? section=2.4& op=?articalID:4811>

How Git Facilitates Collaboration



- **Parallel Development:** Developers can work on different features or bugs in parallel, on separate branches, without disrupting the main codebase.
- **Feature Isolation:** Features or experiments can be developed in isolation and merged back into the main branch only when they are ready, ensuring stability.
- **Code Reviews and Pull Requests:**
Git's workflow supports code reviews and pull requests, where changes can be reviewed and discussed before being integrated into the main codebase.
- **Flexible Workflows:** Git supports various workflows (e.g., Centralized Workflow, Feature Branch Workflow, Forking Workflow), allowing teams to choose the one that best fits their needs.
- **Conflict Resolution:** Git provides tools and strategies for resolving merge conflicts, making it easier for teams to integrate changes smoothly.

What is GitHub?



- What is Git?
a distributed **revision control system**
 - Local
 - Centralized
 - Distributed
- A web-based hosting service for software development projects that used the Git revision control system.
- Provides social networking functionality

Why Git?



1. **Speed and Performance:** Git is designed to handle large projects and large numbers of files with speed and efficiency.
2. **Flexibility:** Git's branching and merging model is powerful and flexible, making it suitable for a wide range of project sizes and team structures.
3. **Distributed Development:** Git's distributed nature is particularly well-suited for remote teams and open-source projects.
4. **Large Community and Ecosystem:** Git has a vast community and a rich ecosystem of tools and services (e.g., GitHub, GitLab, Bitbucket), providing extensive support and resources.
5. **Open Source:** Being open source, Git benefits from continuous improvement and innovation from its community.

Setting Up Git for the First Time



- To start using Git, you first need to install it on your computer and then configure it with your personal details.
- Steps:
 - 1. Installation:** Git is available for all major operating systems.
 - 2. Configuration:** After installing Git, the next step is to configure it with your personal details.

Windows:



1. Download the latest Git for Windows installer from the official Git website or GitHub.
2. Run the installer and follow the on-screen instructions.
3. Choose the components you want to install. For most users, the default options are sufficient.
4. Adjust the PATH environment variable if prompted. It's recommended to select "Use Git from the Windows Command Prompt" to allow using Git from the standard command prompt.
5. Choose a default editor if prompted. You can stick with the default or select your preferred text editor.
6. Configure line endings.
For cross-platform compatibility, it's recommended to select "Checkout as-is, commit Unix-style line endings".
7. Choose how Git should treat file system operations. The default options are usually fine for most users.
8. Finish the installation and launch Git Bash or use the Windows Command Prompt to start using Git.



- The easiest way to install Git on macOS is through the Homebrew package manager.
- If you don't have Homebrew installed, you can install it by running
`/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"` in the Terminal.

- Once Homebrew is installed, run in the Terminal to install Git.

brew install git

- Verify the installation by running
git --version.

Linux



- Git is available in the default repositories of most Linux distributions.
- To install Git on Ubuntu or Debian-based systems, run
sudo apt-get install git
- For Fedora or Red Hat-based systems, use
sudo yum install git or **sudo dnf install git**
- Verify the installation by running
git --version

Configuration



- Set your username:

```
git config --global user.name "Your Name"
```

Replace "Your Name" with your full name.

- Set your email address:

```
git config --global user.email "your\_email@example.com"
```

Replace "your_email@example.com" with the email address you want to associate with your commits.

- (Optional) Set the default text editor:

```
git config --global core.editor "code --wait" # For Visual Studio Code
```

```
git config --global core.editor "atom --wait" # For Atom
```

```
git config --global core.editor "subl -n -w" # For Sublime Text
```

```
git config --global core.editor "vim" # For Vim
```

Replace the command with your preferred text editor.

- (Optional) Check configuration:

```
git config --list # View settings
```

Your Identity

```
wilson@WILSONTHINKPAD /d/temp/gitdemo/aaa <master>
$ git config --global user.name 'Wilson Wang'

wilson@WILSONTHINKPAD /d/temp/gitdemo/aaa <master>
$ git config --global user.email 'wilson@boobooke.com'
```

- Keep track of who worked on the code
- Name
- Email
- Example

```
$ git config --global user.name "John Doe"
```

```
$ git config --global user.email  
johndoe@example.com
```

- Identify is saved in *.gitconfig*
 - Under the home folder

```
wilson@WILSONTHINKPAD ~
$ ls -la
total 6564
drwxr-xr-x 30 wilson Administ 8192 Oct 30 22:56 .
dr-xr-xr-x  8 wilson Administ 4096 Apr 15 2011 ..
-rw-r--r--  1 wilson Administ 6278 Oct 30 22:45 .bash_history
drwxr-xr-x  5 wilson Administ 4096 Jul 11 16:35 .designer
-rw-r--r--  1 wilson Administ 274 Oct 30 22:56 .gitconfig
-rw-r--r--  1 wilson Administ 938 Oct 28 20:15 .gitk
drwxr-xr-x  2 wilson Administ  0 Oct 11 20:23 .ssh
-rw-r--r--  1 wilson Administ 2000655 May  3 23:20 000_0017.JPG
drwxr-xr-x  3 wilson Administ 4096 Aug  6 20:48 Adobe Photoshop CS5.1
```

Cat .gitconfig

- Show the configuration information

```
MINGW32:~
$ cat .gitconfig
[gui]
    recentrepo = D:/WTL/git
[user]
    name = Wilson Wang
    email = wilson@boobooke.com
[alias]
    google = https://wilsonbbk88@code.google.com/p/myboo/
    ci = commit
    last = log -1 HEAD
[github]
    user = Wilson Wang
    token = ff7d7bdeb0b6ef7cd243eb3b81618cc5
[core]
    editor = vi

wilson@WILSONTHINKPAD ~
$
```

Initializing a Repository in an Existing Directory

- Create a folder
- Inside the folder
- `$ git init`

```
wilson@WILSONTHINKPAD /d/temp/gitdemo/aaa
$ ls

wilson@WILSONTHINKPAD /d/temp/gitdemo/aaa
$ ls -l
total 0

wilson@WILSONTHINKPAD /d/temp/gitdemo/aaa
$ vi main.c

wilson@WILSONTHINKPAD /d/temp/gitdemo/aaa
$ cat main.c

int main()
{
    return 0;
}

wilson@WILSONTHINKPAD /d/temp/gitdemo/aaa
$ ls -l
total 1
-rw-r--r--  1 wilson  Administ    34 Oct 30 22:53 main.c

wilson@WILSONTHINKPAD /d/temp/gitdemo/aaa
$ git init
Initialized empty Git repository in d:/temp/gitdemo/aaa/.git/

wilson@WILSONTHINKPAD /d/temp/gitdemo/aaa <master>
$ _
```

.git Folder

```
MINGW32:/d/temp/gitdemo/aaa

wilson@WILSONTHINKPAD /d/temp/gitdemo/aaa <master>
$ ls -l
total 1
-rw-r--r--  1 wilson  Administ      34 Oct 30 22:53 main.c

wilson@WILSONTHINKPAD /d/temp/gitdemo/aaa <master>
$ ls -la
total 3
drwxr-xr-x  3 wilson  Administ       0 Oct 30 22:53 .
drwxr-xr-x  3 wilson  Administ       0 Oct 30 22:52 ..
drwxr-xr-x  6 wilson  Administ    4096 Oct 30 22:53 .git
-rw-r--r--  1 wilson  Administ      34 Oct 30 22:53 main.c

wilson@WILSONTHINKPAD /d/temp/gitdemo/aaa <master>
$
```

- Init command creates .git folder
- Local repository
- Invisible folder
- To display invisible folders

`$ ls -la`



Inside the .git folder

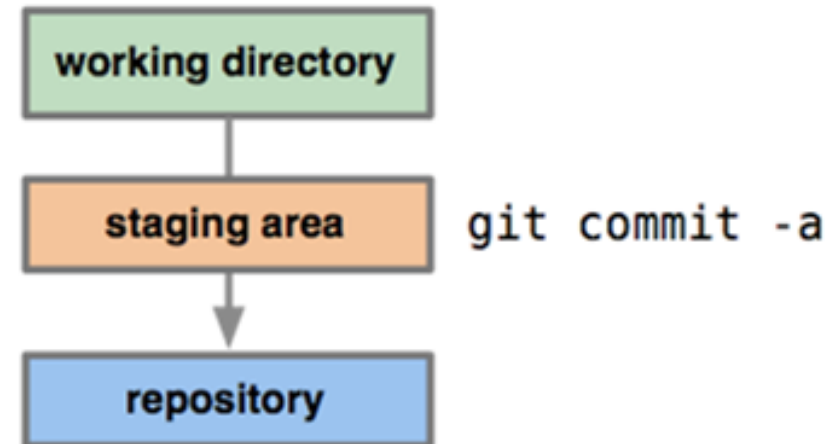
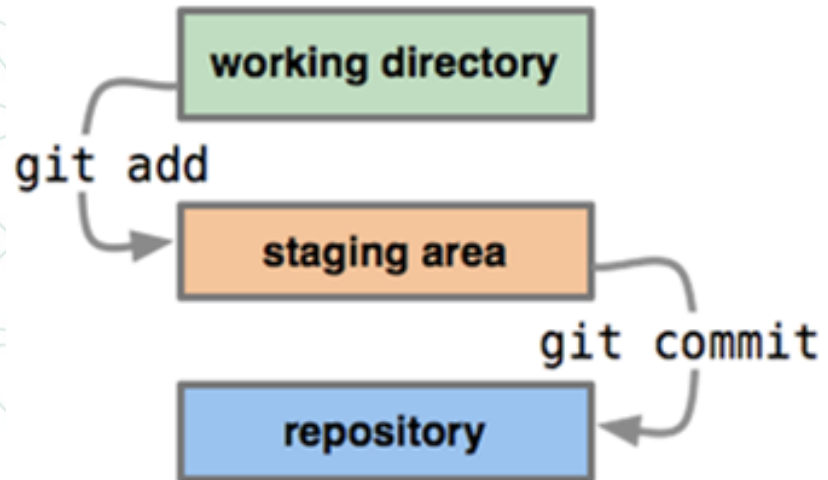
```
wilson@WILSONTHINKPAD /d/temp/gitdemo/aaa (master)
$ ls -la
total 3
drwxr-xr-x  3 wilson  Administ  0 Oct 30 22:53 .
drwxr-xr-x  3 wilson  Administ  0 Oct 30 22:52 ..
drwxr-xr-x  6 wilson  Administ 4096 Oct 30 22:53 .git
-rw-r--r--  1 wilson  Administ  34 Oct 30 22:53 main.c

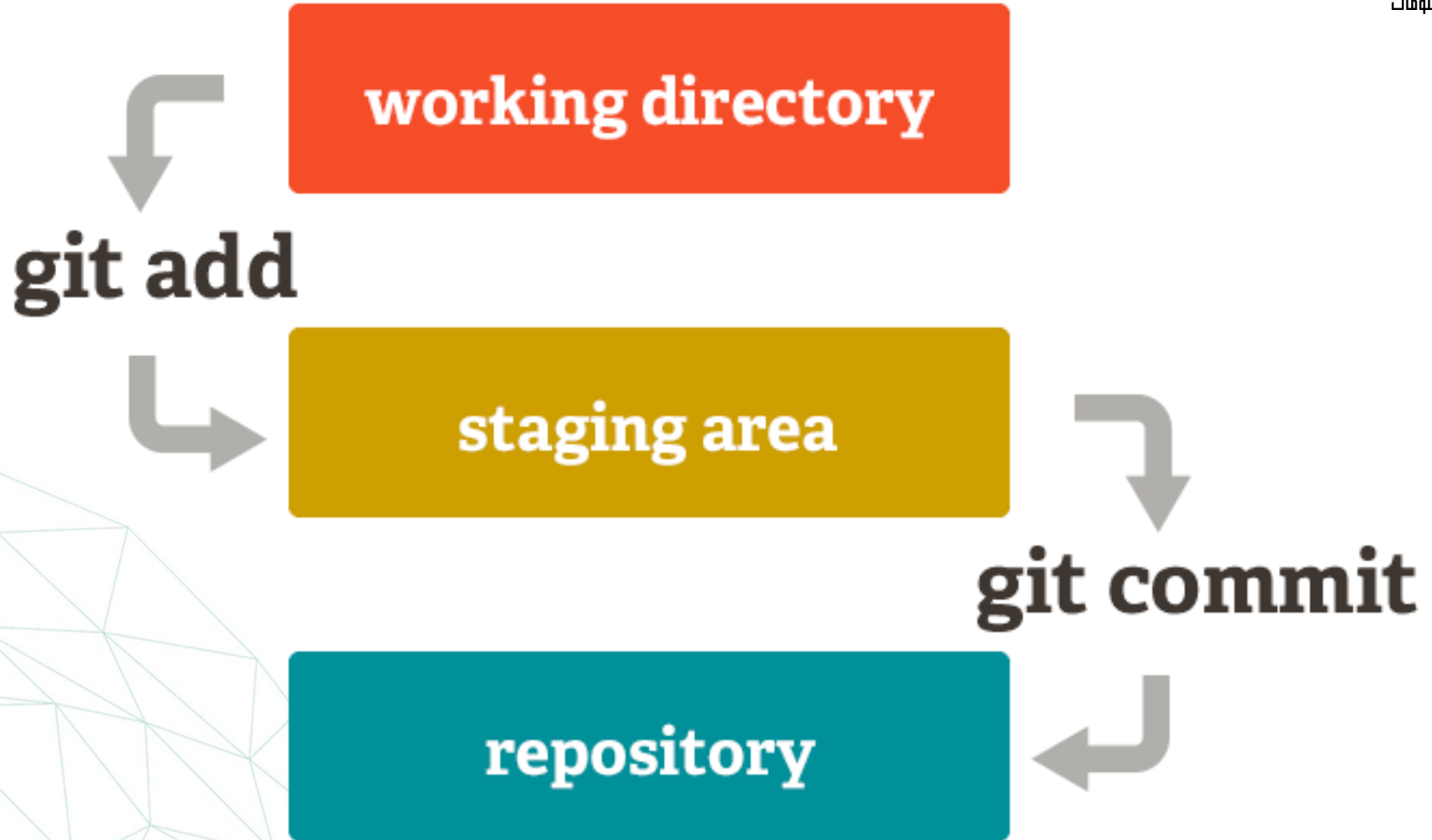
wilson@WILSONTHINKPAD /d/temp/gitdemo/aaa (master)
$ ls -l .git
total 4
-rw-r--r--  1 wilson  Administ  23 Oct 30 22:53 HEAD
-rw-r--r--  1 wilson  Administ 157 Oct 30 22:53 config
-rw-r--r--  1 wilson  Administ  73 Oct 30 22:53 description
drwxr-xr-x  2 wilson  Administ 4096 Oct 30 22:53 hooks
drwxr-xr-x  2 wilson  Administ  0 Oct 30 22:53 info
drwxr-xr-x  4 wilson  Administ  0 Oct 30 22:53 objects
drwxr-xr-x  4 wilson  Administ  0 Oct 30 22:53 refs
```

\$ ls -l .git

Git : Three Areas

- Repository.
- Working directory
- Staging area / Index





Git: Three Areas

- The Git **directory** is where Git stores the metadata and object **database** for your project. This is the most important part of Git, and it is what is copied when you clone a repository from another computer.
- The **working directory** is a single checkout of one version of the project. These files are pulled out of the compressed database in the Git directory and placed on disk for you to use or modify.
- The **staging area** is a simple file, generally contained in your Git directory, that stores information about what will go into your next commit. It's sometimes referred to as the index, but it's becoming standard to refer to it as the staging area.

Basic Git Commands: Add, Commit, Push, and Pull



- Understanding and mastering the basic Git commands is essential for anyone working with Git, as these commands form the core of version control operations.
- Commands:
 - ✓ Add
 - ✓ Commit
 - ✓ Push
 - ✓ Pull

Add

- The **git add** command is used to **stage changes for the next commit**.
- Staging means that you are preparing files for Git to take a snapshot of the staged version and save it in the commit history.
- This command doesn't actually upload anything to the remote repository; it just updates the local index.

git add <file>

<file> can be a single file or a pattern that matches multiple files.

For example,

git add .

stages all changes in the current directory and its subdirectories.

Commit



- The **git commit** command is used to save your changes to the local repository.
- It takes all of the changes written in the index (**staging area**), creates a new commit object pointing to it, and sets the branch to point to that commit.
- Think of it as a snapshot of your repository at that point in time.

git commit -m "Commit message"

-m allows you to add a commit message inline.

It's important to write a clear and descriptive commit message that explains what changes you made and why.

Adding and Committing

```
MINGW32:/d/temp/gitdemo/aaa

wilson@WILSONTHINKPAD /d/temp/gitdemo/aaa (master)
$ pwd
/d/temp/gitdemo/aaa

wilson@WILSONTHINKPAD /d/temp/gitdemo/aaa (master)
$ ls -l
total 1
-rw-r--r--  1 wilson  Administ      34 Oct 30 22:53 main.c

wilson@WILSONTHINKPAD /d/temp/gitdemo/aaa (master)
$ git add main.c

wilson@WILSONTHINKPAD /d/temp/gitdemo/aaa (master)
$ git commit -m "Init Project"
[master (root-commit) fdafc20] Init Project
1 files changed, 6 insertions(+), 0 deletions(-)
create mode 100644 main.c
```

Staging Area

```
wilson@WILSONTHINKPAD /d/temp/gitdemo/bbb <master>
$ ls -l .git
total 4
-rw-r--r-- 1 wilson Administ 23 Oct 31 22:37 HEAD
-rw-r--r-- 1 wilson Administ 157 Oct 31 22:37 config
-rw-r--r-- 1 wilson Administ 73 Oct 31 22:37 description
drwxr-xr-x 2 wilson Administ 4096 Oct 31 22:37 hooks
drwxr-xr-x 2 wilson Administ 0 Oct 31 22:37 info
drwxr-xr-x 4 wilson Administ 0 Oct 31 22:37 objects
drwxr-xr-x 4 wilson Administ 0 Oct 31 22:37 refs

wilson@WILSONTHINKPAD /d/temp/gitdemo/bbb <master>
$ git add main.c

wilson@WILSONTHINKPAD /d/temp/gitdemo/bbb <master>
$ ls -l .git
total 4
-rw-r--r-- 1 wilson Administ 23 Oct 31 22:37 HEAD
-rw-r--r-- 1 wilson Administ 157 Oct 31 22:37 config
-rw-r--r-- 1 wilson Administ 73 Oct 31 22:37 description
drwxr-xr-x 2 wilson Administ 4096 Oct 31 22:37 hooks
-rw-r--r-- 1 wilson Administ 104 Oct 31 22:38 index
drwxr-xr-x 2 wilson Administ 0 Oct 31 22:37 info
drwxr-xr-x 5 wilson Administ 0 Oct 31 22:38 objects
drwxr-xr-x 4 wilson Administ 0 Oct 31 22:37 refs
```

Before and after *git add*

Push



- The **git push** command is used to upload local commits to a remote repository.
- Pushing lets you move your changes to a remote repository, which is typically hosted on a service like GitHub, GitLab, or Bitbucket.

git push <remote> <branch>

<remote> is the name of the remote repository you want to push to, usually **origin**.

<branch> is the branch you want to push to, commonly **main** or **master**.

Pull



- The **git pull** command is used to **fetch and merge** changes from a remote repository to your local repository.
It's a combination of **git fetch** followed by **git merge**.
- This command is useful when you want to update your local repository with the latest changes from a remote repository.

git pull <remote> <branch>

<remote> is the name of the remote repository.

<branch> is the branch you want to merge into your current branch.

Navigating the GitHub Interface

- **GitHub** is a web-based platform that provides hosting for software development and version control using Git.
- It offers a user-friendly interface that allows developers to collaborate on projects, manage code repositories, and track issues.
- Familiarizing yourself with the GitHub interface is essential for leveraging its full potential.
- The GitHub interface is designed to be intuitive, with key features easily accessible from the main dashboard.
- **Dashboard:** This is your landing page after logging in. It provides an overview of your activity feed, showing recent commits, pull requests, issues, and other relevant updates from repositories you're watching or contributing to.
- **Repositories:** This section lists all the repositories you own or have access to.
You can navigate to a specific repository to view its files, commit history, branches, and more.
- **Pull Requests:** This feature allows you to contribute to projects by proposing changes to the repository.
You can review, comment on, and merge pull requests directly from the interface.
- **Issues:** GitHub's issue tracking system lets you discuss and keep track of bugs, enhancements, and tasks within a repository. You can create new issues, assign them to collaborators, and organize them with labels and milestones.
- **Marketplace & Explore:** These sections allow you to discover new tools, actions, and apps to extend GitHub's functionality, as well as explore other open-source projects and find inspiration.

Creating an Account



steps to do so:

- 1) Visit GitHub: Go to GitHub in your web browser.
- 2) Sign Up: Click on the "Sign up" button in the top-right corner of the page.
- 3) Choose a Plan: GitHub offers various plans, including a free personal plan that is suitable for most individual users and small teams. Select the plan that best fits your needs.
- 4) Enter Your Details: Fill in the required information, including your email address, username, password, and optional fields like your name and location.
- 5) Verify Your Account: After submitting your details, you'll need to verify your email address. Check your inbox for a message from GitHub and follow the instructions to verify your account.
- 6) Set Up Your Profile: Once your account is verified, you can set up your profile by adding a profile picture, bio, and other information to make it more professional and informative.
- 7) Start Using GitHub: With your account set up, you can now create your first repository, start collaborating on projects, and explore the vast array of features GitHub has to offer.
- 8) By following these steps and familiarizing yourself with the GitHub interface, you'll be well on your way to leveraging GitHub's powerful tools for code collaboration, version control, and project management.

Creating Repositories



- **Steps:**
 - 1) **Log in to GitHub:** Visit the GitHub website and log in to your account.
 - 2) **Access the Repositories Page:** Click on the "+" icon in the top-right corner of the page and select "New repository" from the dropdown menu.
 - 3) **Repository Name:** Give your repository a name. The name should be concise and descriptive of the project's content.
 - 4) **Description (Optional):** You can add a description of your repository. This is helpful for others to understand the purpose of your project.
 - 5) **Public/Private:** Choose between making your repository public or private. Public repositories are visible to everyone, while private repositories are only accessible to you or collaborators you choose.
 - 6) **Initialize with README (Optional):** You have the option to initialize your repository with a README file, .gitignore, and a license. Initializing with a README will make your repository immediately available with a home page.
 - 7) **Create Repository:** Click the "Create repository" button to finalize the setup.
- Once created, you can start adding files, folders, and commits to your new repository directly on GitHub or by cloning it to your local machine.

Cloning Repositories

- Cloning a repository means creating a local copy of a GitHub repository on your computer. This allows you to work on the project's files locally and then push your changes back to GitHub.

How to clone a repository:

- 1) **Locate the Repository:** Navigate to the repository you wish to clone on GitHub.
 - 2) **Clone Button:** Click the "Code" button (previously labeled as "Clone or download") and copy the URL provided. This URL is your repository's clone URL and allows Git to find and duplicate your repository.
 - 3) **Choose a Directory:** Open your terminal (or Git Bash on Windows) and use the `cd` command to navigate to the directory where you want to clone the repository.
 - 4) **Clone the Repository:** Use the `git clone` command followed by the URL you copied:
 - 5) `git clone https://github.com/username/repository-name.git`
 - 6) Replace `https://github.com/username/repository-name.git` with the URL of the repository you want to clone.
 - 7) **Wait for Cloning to Complete:** Git will now clone the repository to your local machine. This process might take some time depending on the size of the repository.
- Once the cloning process is complete, you'll have a full copy of the repository on your local machine, and you can start working on the project.
 - Remember to regularly push your changes back to GitHub to keep the remote repository updated with your local changes.

Quiz Time

Key Takeaways

- **Version Control:** Tracks and manages code changes.
- **Git:** A distributed VCS created by Linus Torvalds.
- **GitHub:** A cloud platform that hosts Git repositories.
- **Basic Workflow:** add → commit → push → pull
- **Configuration:** Use git config to set your identity.

THANK YOU

